

Implementación de Acceso a Medios

Juan Jose Duarte Urrego

Institucion Universitaria Pacual Bravo

Sistemas Operativos

Jonathan Berthel Castro

6/04/2026

INFORME TÉCNICO

Objetivo del programa.

El propósito de esta implementación es garantizar la persistencia de datos críticos del SIGET, simulando el almacenamiento de métricas (mediante el sensor de aceleración MPU6050) en un entorno limitado de microcontrolador. Se implementan estrategias para leer el sensor y escribir en una Micro-SD virtual, a la vez que se replica la lógica de acceso aleatorio y secuencial en un entorno como C++ para el procesamiento de logs, demostrando las capacidades requeridas en una infraestructura de semaforización inteligente.

Lógica de implementación

Lógica de implementación : Se abre un archivo nuevo en modo de escritura (FILE_WRITE o ios::out) para plasmar el encabezado del registro. Actualización o sea, Aleatoria con Desplazamiento, dado que tanto SD.h como los flujos de C++ sobrescriben caracteres al situarse en posiciones específicas de archivos de texto plano, la lógica adoptada consistió en utilizar la función seek() / seekg() para calcular tamaños, y cargar el contenido adyacente en memoria, se realiza la inserción en la posición 10 y luego se reescribe la cadena completa para evitar la sobrescritura destructiva de los datos subsecuentes, luego en la normalización (Búsqueda in-situ): Aprovechando el contenido cargado en el búfer temporal, se recorrió el archivo en búsqueda de los caracteres 'e' reemplazándolos de forma directa por el número '3', minimizando los accesos al disco físico antes de guardar. Alerta (Secuencial Adicional) al final se empleó un posicionamiento en el final del archivo (apertura en modo append en C++) para escribir una alerta crítica sin afectar los registros de aceleración previamente calculados por el sensor.

Funciones y parámetros

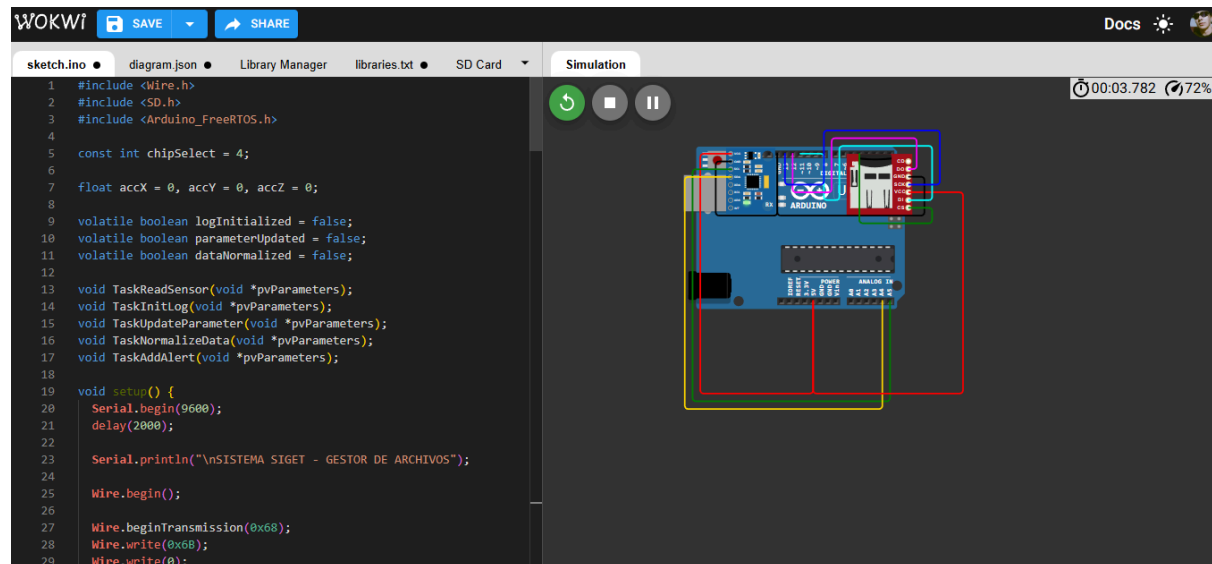
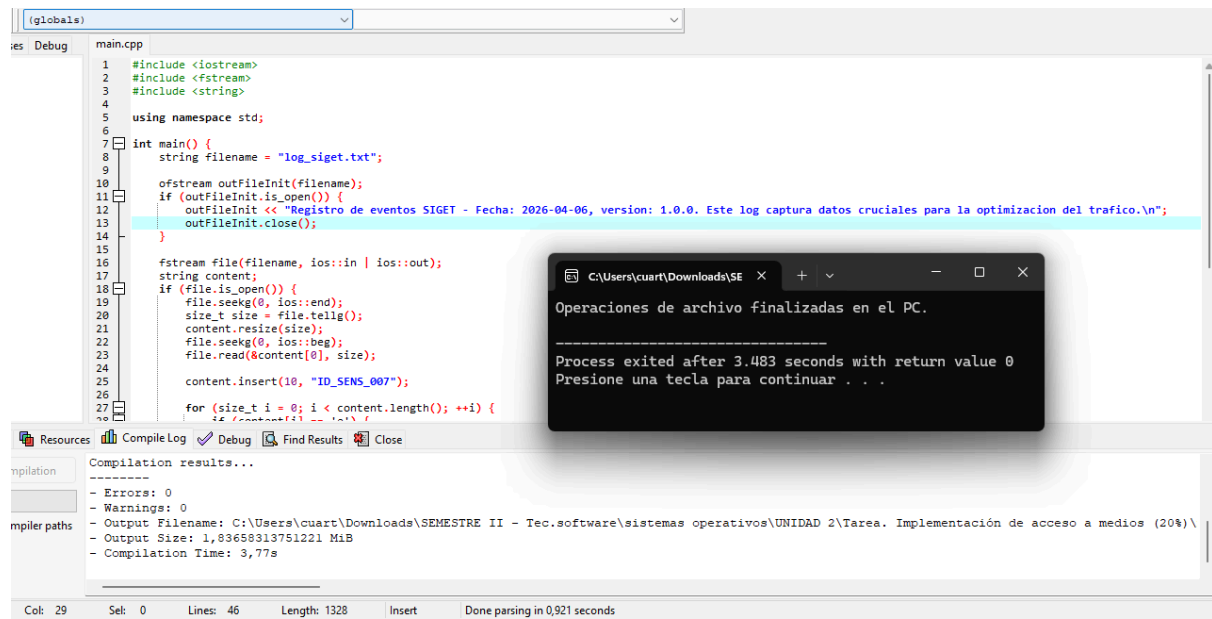
Funciones y parámetros SD.open() / fstream: se utilizó para abrir el archivo, asignándole modos de lectura o escritura según el ciclo.file.write() / file.print() / ostream<<: Implementados para inyectar datos (cadenas y variables float del MPU6050) en la tarjeta de memoria.file.seek() / seekg() / seekp(): Operaciones ejecutadas para posicionar el puntero interno tanto de lectura como de escritura, habilitando el dinamismo en la manipulación aleatoria.file.close(): Ejecutado indefectiblemente tras cada bloque de operaciones para descargar los buffers a disco y asegurar la integridad del archivo.FreeRTOS (xTaskCreate): Empleado en Arduino para levantar dos tareas concurrentes, distribuyendo el tiempo del procesador (Time Slicing).

Relación con la asignatura

Relación con la asignatura: en esta actividad se demuestran en la práctica las abstracciones de hardware que provee un sistema operativo. En primer lugar, la gestión de la Micro-SD refleja con exactitud cómo un SO administra la tabla de asignación y el sistema de archivos del disco físico. Así mismo, el manejo de sensores y almacenamiento ejemplifica de forma directa la interacción de un sistema con los dispositivos periféricos de Entrada y Salida (E/S). Por último, el orquestador FreeRTOS simula el planificador de un SO moderno al encargarse de la gestión de tareas o procesos , asegurando una concurrencia eficiente y previniendo que los ciclos del I2C (sensor) y del SPI (SD) interfieran entre sí al compartir los recursos del microcontrolador.

ACCESO A MEDIOS

EVIDENCIA



Texto creado

